In [23]:
```
# data manipulation
import pandas as pd
import numpy as np

#import attricion data
df=pd.read_csv('C:\\Users\\Public\\Desktop\\DAT-430\\HR Attrition Data.csv')
```

In [24]:
```
df['EmployeeNumber'].head
```

Out[24]:
```
<bound method NDFrame.head of 0        282
1          2
2       1082
3          5
4          7
        ...
1465    2061
1466    2062
1467    2064
1468    2065
1469    2068
Name: EmployeeNumber, Length: 1470, dtype: int64>
```

In [2]:
```
#remove columns
df.drop(['DailyRate',
        'EmployeeCount',
        'EmployeeNumber',
        'Over18'], axis=1, inplace=True)
```

In [3]:
```
#changing Attrition from object to numbers

df['Attrition'] = df['Attrition'].map({'No': 0, 'Yes': 1})
```

In [4]:
```
#checking unique value for later binarization
print("BusinessTravel:", sorted(df['BusinessTravel'].unique()))
print("Department:", sorted(df['Department'].unique()))
print("EducationField:", sorted(df['EducationField'].unique()))
print("Gender:", sorted(df['Gender'].unique()))
print("JobRole:", sorted(df['JobRole'].unique()))
print("MaritalStatus:", sorted(df['MaritalStatus'].unique()))
print("OverTime:", sorted(df['OverTime'].unique()))
```

```
BusinessTravel: ['Non-Travel', 'Travel_Frequently', 'Travel_Rarely']
Department: ['Human Resources', 'Research & Development', 'Sales']
EducationField: ['Human Resources', 'Life Sciences', 'Marketing', 'Medic
al', 'Other', 'Technical Degree']
Gender: ['Female', 'Male']
JobRole: ['Healthcare Representative', 'Human Resources', 'Laboratory Te
chnician', 'Manager', 'Manufacturing Director', 'Research Director', 'Re
search Scientist', 'Sales Executive', 'Sales Representative']
MaritalStatus: ['Divorced', 'Married', 'Single']
OverTime: ['No', 'Yes']
```

In [5]:
```
#binarization
df['OverTime'] = df['OverTime'].map({'No': 0, 'Yes': 1})
df['BusinessTravel'] = df['BusinessTravel'].map({"Non-Travel":0, "Travel
_Rarely":1, "Travel_Frequently":2})
df['Department']=df['Department'].map({'Human Resources':0, 'Research &
Development':1, 'Sales':2})
df['JobRole']=df['JobRole'].map({'Healthcare Representative':0, 'Human R
esources':1, 'Laboratory Technician':2, 'Manager':3, 'Manufacturing Dire
ctor':4, 'Research Director':5, 'Research Scientist':6, 'Sales Executiv
e':7, 'Sales Representative':8})
df['EducationField']=df['EducationField'].map({'Human Resources':0, 'Lif
e Sciences':1, 'Marketing':2, 'Medical':3, 'Other':4, 'Technical Degre
e':5})
df['Gender']=df['Gender'].map({'Female':0, 'Male':1})
df['MaritalStatus']=df['MaritalStatus'].map({'Divorced':2, 'Married':1,
'Single':0})
```

In [11]:

In [6]:
```
from sklearn.feature_selection import RFE
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

X = df.drop('Attrition', axis=1)
y = df['Attrition']

# Assume X is your feature matrix, y is the target variable
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, stratify=y, random_state=42
)

model = LogisticRegression(max_iter=1000)
rfe = RFE(estimator=model, n_features_to_select=10)  # Choose the number of features
X_rfe = rfe.fit_transform(X_train, y_train)

print("Selected features:", X.columns[rfe.support_])
```

```
C:\ProgramData\Anaconda3\lib\site-packages\sklearn\linear_model\_logisti
c.py:762: ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown
in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-r
egression
  n_iter_i = _check_optimize_result(
C:\ProgramData\Anaconda3\lib\site-packages\sklearn\linear_model\_logisti
c.py:762: ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown
in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-r
egression
  n_iter_i = _check_optimize_result(

Selected features: Index(['BusinessTravel', 'JobLevel', 'JobSatisfactio
n', 'OverTime',
       'PerformanceRating', 'RelationshipSatisfaction', 'StockOptionLeve
l',
       'WorkLifeBalance', 'YearsSinceLastPromotion', 'YearsWithCurrManag
er'],
      dtype='object')
```

In [ ]:

In [7]:
```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier()
rfe = RFE(estimator=model, n_features_to_select=10)
X_rfe = rfe.fit_transform(X_train, y_train)

print("Selected features:", X.columns[rfe.support_])
```

```
Selected features: Index(['Age', 'JobRole', 'JobSatisfaction', 'MonthlyI
ncome',
       'PerformanceRating', 'TotalWorkingYears', 'WorkLifeBalance',
       'YearsAtCompany', 'YearsInCurrentRole', 'YearsWithCurrManager'],
      dtype='object')
```

In [8]:
```
#apply logistic regression model

selected_features = ['BusinessTravel', 'JobLevel', 'JobSatisfaction', 'O
verTime', 'RelationshipSatisfaction', 'StockOptionLevel',
        'YearsSinceLastPromotion', 'YearsWithCurrManager']

X = df[selected_features]
y = df['Attrition']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, stratify=y, random_state=42
)

from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, accuracy_score, confu
sion_matrix

logreg = LogisticRegression(max_iter=1000)
logreg.fit(X_train, y_train)

y_pred_logreg = logreg.predict(X_test)

print("Logistic Regression Results:")
print("Accuracy:", accuracy_score(y_test, y_pred_logreg))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_logreg))
print("Classification Report:\n", classification_report(y_test, y_pred_l
ogreg))
```

```
Logistic Regression Results:
Accuracy: 0.9478458049886621
Confusion Matrix:
 [[356  14]
 [  9  62]]
Classification Report:
              precision    recall  f1-score   support

           0       0.98      0.96      0.97       370
           1       0.82      0.87      0.84        71

    accuracy                           0.95       441
   macro avg       0.90      0.92      0.91       441
weighted avg       0.95      0.95      0.95       441
```

In [9]:
```
#apply random forest
selected_features = ['Age', 'JobLevel', 'JobRole', 'JobSatisfaction', 'M
onthlyIncome',
        'TotalWorkingYears', 'Department',
       'YearsAtCompany', 'YearsWithCurrManager']

X = df[selected_features]
y = df['Attrition']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, stratify=y, random_state=42
)

from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)

y_pred_rf = rf.predict(X_test)

print("Random Forest Results:")
print("Accuracy:", accuracy_score(y_test, y_pred_rf))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_rf))
print("Classification Report:\n", classification_report(y_test, y_pred_r
f))
```

```
Random Forest Results:
Accuracy: 0.9455782312925171
Confusion Matrix:
 [[353  17]
 [  7  64]]
Classification Report:
              precision    recall  f1-score   support

           0       0.98      0.95      0.97       370
           1       0.79      0.90      0.84        71

    accuracy                           0.95       441
   macro avg       0.89      0.93      0.90       441
weighted avg       0.95      0.95      0.95       441
```

In [10]:
```
df.drop(['HourlyRate',
        'StandardHours',
        'TrainingTimesLastYear'
        ], axis=1, inplace=True)
```

In [11]:
```
df.drop(['EducationField',
        'Education',
        'DistanceFromHome',
         'PerformanceRating'
        ], axis=1, inplace=True)
```

In [12]:
```
df.head()
```

Out[12]:

	Age	Attrition	BusinessTravel	Department	EnvironmentSatisfaction	Gender	JobInvolve
0	38	1	1	1	2	1	
1	49	0	2	1	3	1	
2	28	1	0	1	2	1	
3	33	0	2	1	4	0	
4	27	0	1	1	1	1	

5 rows × 24 columns

In [ ]:
```
```

In [13]:
```
selected_features = ['BusinessTravel', 'JobLevel', 'JobSatisfaction', 'O
verTime', 'RelationshipSatisfaction', 'StockOptionLevel',
        'YearsSinceLastPromotion', 'YearsWithCurrManager']

#making predictions logistic regression
employee = pd.DataFrame([{
    'BusinessTravel':1, 'JobLevel':0, 'JobSatisfaction':1, 'OverTime':1,
'RelationshipSatisfaction':2, 'StockOptionLevel':0,
        'YearsSinceLastPromotion':0, 'YearsWithCurrManager':0

}])

# Predict whether they will leave (0 = stay, 1 = attrition)
prediction = logreg.predict(employee[selected_features])
probability = logreg.predict_proba(employee[selected_features])[:, 1]

print("Prediction:", prediction[0])  # 0 or 1
print("Probability of Attrition:", round(probability[0], 2))
```

```
Prediction: 1
Probability of Attrition: 0.98
```

In [14]:
```
df.head()
```

Out[14]:

	Age	Attrition	BusinessTravel	Department	EnvironmentSatisfaction	Gender	JobInvolve
0	38	1	1	1	2	1	
1	49	0	2	1	3	1	
2	28	1	0	1	2	1	
3	33	0	2	1	4	0	
4	27	0	1	1	1	1	

5 rows × 24 columns

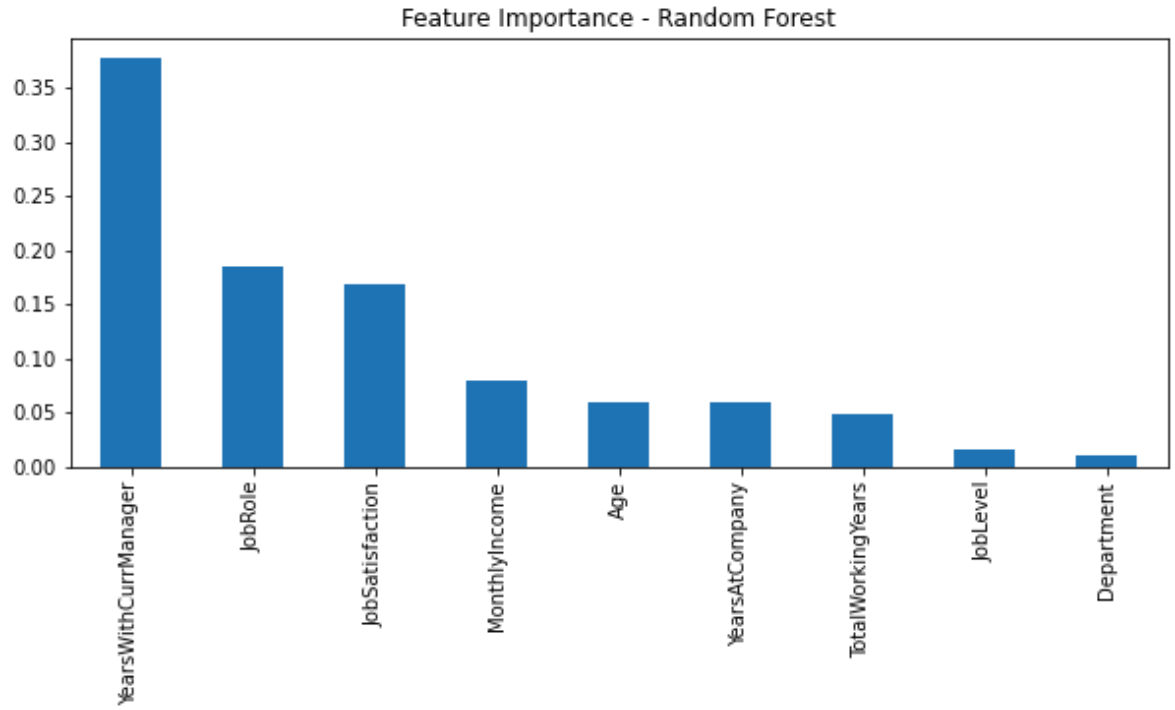
In [15]:
```
df['MonthlyIncome'].head
```

Out[15]:
```
<bound method NDFrame.head of 0       6673
1       5130
2       8722
3       2909
4       3468
        ...
1465    2571
1466    9991
1467    6142
1468    5390
1469    4404
Name: MonthlyIncome, Length: 1470, dtype: int64>
```

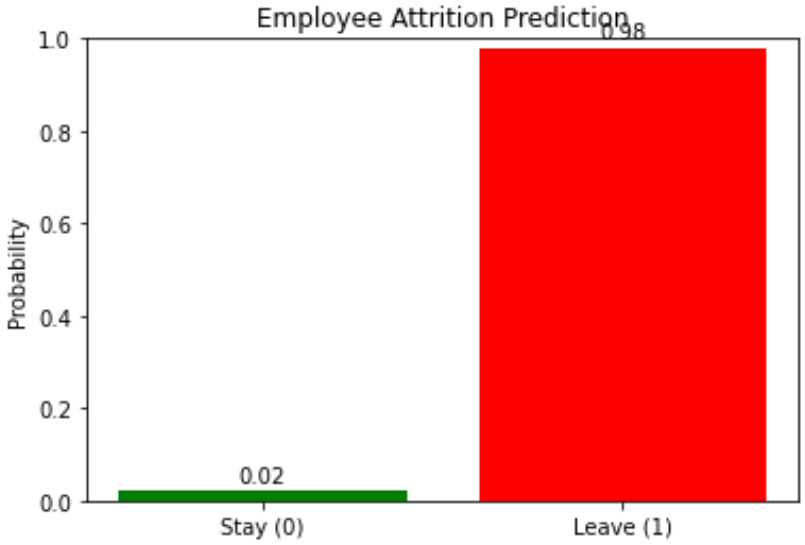
In [16]:
```
selected_features = ['Age', 'JobLevel', 'JobRole', 'JobSatisfaction', 'M
onthlyIncome',
        'TotalWorkingYears', 'Department',
       'YearsAtCompany', 'YearsWithCurrManager']


#making predictions logistic regression
employee = pd.DataFrame([{
    'Age':49, 'JobLevel':2, 'JobRole':6, 'JobSatisfaction':2, 'MonthlyIn
come':8722,
        'TotalWorkingYears':10, 'Department':1,
       'YearsAtCompany':10, 'YearsWithCurrManager':7

}])

# Predict class (0 = stay, 1 = attrition)
prediction = rf.predict(employee[selected_features])

# Predict probability of attrition
probability = rf.predict_proba(employee[selected_features])[:, 1]

print("Random Forest Prediction:", prediction[0])
print("Probability of Attrition:", round(probability[0], 2))
```

```
Random Forest Prediction: 0
Probability of Attrition: 0.0
```

In [89]:
```
import pandas as pd
import matplotlib.pyplot as plt

importances = rf.feature_importances_
feat_importance = pd.Series(importances, index=selected_features).sort_v
alues(ascending=False)

feat_importance.plot(kind='bar', figsize=(10, 4), title='Feature Importa
nce - Random Forest')
plt.show()
```



In [17]:
```
import matplotlib.pyplot as plt

labels = ['Stay (0)', 'Leave (1)']
probs = [1 - 0.98, 0.98]

plt.bar(labels, probs, color=['green', 'red'])
plt.title('Employee Attrition Prediction')
plt.ylabel('Probability')
plt.ylim(0, 1)
plt.text(0, probs[0] + 0.02, f'{probs[0]:.2f}', ha='center')
plt.text(1, probs[1] + 0.02, f'{probs[1]:.2f}', ha='center')
plt.show()
```



In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [17]:

In [33]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]: